DATA STRUCTURES IN C
MODULES 1 AND 2

High-yield notes, worked algorithms, and question bank

Exam Focus.Priority order. Master (1) infix to postfix and prefix conversion, (2) postfix evaluation, and (3) sparse-matrix triplets and addition. Then revise stacks/queues, arrays, pointers, dynamic memory, polynomials, and storage classes. For algorithm answers: idea, steps, one trace, complexity, limitation.

Contents

1 Arrays and Linear Structures **2**
- 1.1 Arrays . . . 2
- 1.2 Stack: LIFO . . . 2
- 1.3 Queue: FIFO . . . 2

2 Infix, Prefix, Postfix, and Stack Algorithms **2**
- 2.1 Notation . . . 2
- 2.2 Infix to postfix . . . 2
  - 2.2.1 Worked trace: $A + B * C - D/E$ . . . 3
- 2.3 Infix to prefix . . . 3
- 2.4 Postfix evaluation . . . 3

3 Sparse Matrix Representation and Operations **3**
- 3.1 Triplet representation . . . 3
- 3.2 Conversion algorithm . . . 3
- 3.3 Addition . . . 4

4 Polynomials and Arrays **4**

5 Pointers and Dynamic Memory **4**
- 5.1 Dynamic allocation . . . 4

6 Storage Classes in C **4**

7 Important Exam Topics **4**

8 10-Mark Answer Templates **5**
- 8.1 Expression conversion **[10 marks]** . . . 5
- 8.2 Sparse matrix **[10 marks]** . . . 5

9 Question Bank **5**

10 Last-Minute Revision Page **6**

Arrays and Linear Structures

Arrays

An array stores same-type elements in contiguous memory. Valid indices for `a[n]` are `0` through `n-1`; random access is $O(1)$:

$$\mathrm{LOC}(a[i]) = \mathrm{base}(a) + i \times \mathrm{sizeof}(a[0]).$$

Two-dimensional arrays are commonly row-major. Traversal is $O(n)$; insertion or deletion in the middle is $O(n)$ because elements shift.

> **Remember.Remember:** An array name usually acts like the address of its first element, but an array is not a modifiable pointer.

Stack: LIFO

A stack inserts and deletes only at **TOP**. `push` adds; `pop` removes. Both are $O(1)$ with an array. Overflow means push on a full stack; underflow means pop on an empty stack. Applications: expression conversion/evaluation, recursion, undo, backtracking, and reversal.

Queue: FIFO

A queue inserts at the **rear** and deletes at the **front**. Operations are `enQueue`, `deQueue`, and `display`. With an array, initialize `front = rear = -1`. A circular queue avoids wasted positions in a linear array.

Operation	Action	Result
enQueue(x)	check full; increment rear; store x	enters at rear
deQueue()	check empty; remove front; advance front	leaves at front
display()	scan front through rear	FIFO order

Infix, Prefix, Postfix, and Stack Algorithms

Notation

Notation	Operator position	Example
Infix	between operands	$a + b$
Prefix	before operands	$+ab$
Postfix	after operands	$ab+$

Infix needs precedence and parentheses. Prefix and postfix encode order directly. Precedence is: parentheses, exponentiation, multiplication/division/modulus, addition/subtraction. Equal-precedence operators normally use left associativity; exponentiation is commonly right associative.

Infix to postfix

Scan left to right. Output operands. Push `(`. For `)`, pop until `(`. For an operator, pop higher-precedence operators (and applicable equal-precedence ones), then push the new operator. At the end, pop the stack.

1. Read next token.
2. Operand: append to output.
3. `(`: push.
4. `)`: pop to output through matching `(`.
5. Operator: pop by precedence/associativity, then push.
6. End: pop all remaining operators.

2.2.1 Worked trace: $A + B * C - D/E$

Token	Stack (bottom to top)	Output
A	–	A
+	+	A
B	+	AB
*	+*	AB
C	+*	ABC
-	+-	ABC*
D	+-	ABC*D
/	+-/	ABC*D
E	+-/	ABC*DE
end	–	**ABC*D/E-+**

Exam Focus.Manual shortcut: Fully parenthesize, then move each operator to the right of its operand pair. $A + ((B * C) - (D/E)) \rightarrow ABC * DE/ - +$.

Infix to prefix

1. Reverse the infix expression.
2. Swap `(` and `)`.
3. Convert the modified expression to postfix.
4. Reverse the result.

For $A + B * C - D/E$, prefix is $- + A * BC/DE$.

Postfix evaluation

Push operands. On an operator, pop right operand first, pop left operand second, compute, and push the result. Example: 1 2 3 + *: push $1, 2, 3$; compute $2 + 3 = 5$; compute $1 \times 5 = 5$.

Do Not Confuse.Operand trap: For subtraction/division, popping b then a means calculate $a - b$ or a/b, not the reverse.

Sparse Matrix Representation and Operations

Triplet representation

A sparse matrix has mostly zeros. Store only nonzero entries as $(row, column, value)$; the header can store $(rows, columns, nonzero\ count)$.

	Column					
	0	1	2	3	4	5
0	0	0	4	0	0	5
1	0	3	0	0	0	0
2	0	0	2	0	0	0
3	2	0	0	0	0	0
4	0	0	0	0	0	7

Row	Col	Value
5	6	6
0	2	4
0	5	5
1	1	3
2	2	2
3	0	2
4	5	7

The header $5, 6, 6$ means five rows, six columns, and six nonzero values.

Conversion algorithm

1. Set `k=1`; place dimensions in row 0.
2. Scan every matrix cell row by row.
3. If `a[i][j] != 0`, store $i, j, a[i][j]$ in row `k`; increment `k`.
4. Set the header nonzero count to `k-1`.

Dense storage is $O(rc)$; triplet storage is $O(t)$ for t nonzero terms. Scanning costs $O(rc)$.

Addition

Addition requires equal dimensions. With triplets sorted by row and column, compare current positions: copy the smaller position, or add values when positions match. Store a result only if the sum is nonzero. After sorting, time is $O(t_A + t_B)$.

Answer Skeleton. For a 10-mark answer: define sparse matrix; draw dense and triplet forms; explain the header; write conversion steps and complexity; state the dimension condition; explain merge addition; show one matching position; mention memory savings.

Polynomials and Arrays

A polynomial such as $5x^3 - 2x + 7$ consists of coefficient-exponent terms. Align equal exponents for addition/subtraction. Distribute for multiplication and combine like terms. A polynomial can use an array indexed by exponent, or pairs $(coefficient, exponent)$ when many terms are zero. The pair representation is sparse storage in miniature.

Pointers and Dynamic Memory

A pointer stores an address. `&x` gives the address of `x`; `*p` dereferences `p`.

```
int x = 10; int *p = &x;
printf("%d %p", *p, (void *)p);
```

Array access obeys `a[i] == *(a+i)`. NULL points to no object; a wild pointer is uninitialized; a dangling pointer refers to freed/out-of-scope storage; a void pointer is generic and must be converted before dereferencing.

Dynamic allocation

`malloc` allocates uninitialized bytes; `calloc` allocates and zero-initializes; `realloc` resizes; `free` releases.

```
int *p = malloc(n * sizeof *p);
if (p == NULL) { /* failure */ }
free(p); p = NULL;
```

Use a temporary pointer for `realloc`; otherwise failure may lose the original block. Match every successful allocation with one `free`.

Storage Classes in C

Class	Scope	Lifetime	Key point
auto	block	block; indeterminate if uninitialized	default local class
register	block	block	request fast storage; no portable address
static local	block	whole program	retains value between calls
static global	file	whole program	internal linkage
extern	declaration	defined elsewhere	refers to another file's global/function

Do Not Confuse.Do not confuse: Scope is where a name is visible; lifetime is how long the object exists; linkage is which declarations denote the same object.

Important Exam Topics

- Infix to postfix with a stack trace; infix to prefix by reversal.

- Postfix evaluation with correct left/right operand order.
- Sparse-matrix triplets, conversion, and addition.
- Stack versus queue; push/pop and enqueue/dequeue.
- Pointer basics, pointer arithmetic, and array equivalence.
- `malloc`, `calloc`, `realloc`, and `free`.
- Scope/lifetime/linkage of auto, register, static, and extern.

10-Mark Answer Templates

Expression conversion [10 marks]

Answer Skeleton.
1. Define infix, prefix, postfix.
2. State precedence and associativity.
3. Write the stack algorithm.
4. Show a token table.
5. Explain prefix reversal method.
6. Give and verify final answers.

Sparse matrix [10 marks]

Answer Skeleton.
1. Define sparse matrix and motivation.
2. Draw triplet representation.
3. Give conversion algorithm and complexity.
4. State equal-dimension condition.
5. Explain merge addition and matching entries.
6. Conclude with storage advantage.

Question Bank

Definitions / 2–3 marks

1. Define LIFO and FIFO.
2. Define infix, prefix, postfix.
3. What are overflow and underflow?
4. What is a sparse matrix and a triplet row?
5. Distinguish NULL, wild, and dangling pointers.
6. State the purpose of static and extern.

5-mark questions

1. Convert $A + B * C - D/E$ to postfix with a trace.
2. Convert $A + (B * C - D)/E$ to prefix.
3. Evaluate 1 2 3 $+$ $*$.
4. Convert a matrix to triplets.
5. Compare malloc and calloc; explain safe realloc.
6. Compare array stack and queue.

10-mark questions

1. Explain infix-to-postfix and infix-to-prefix algorithms with examples.
2. Explain sparse triplets and addition.
3. Explain pointers and dynamic allocation.
4. Explain all four storage classes.

Last-Minute Revision Page

STACK
LIFO; push/pop at TOP.

QUEUE
FIFO; insert rear, delete front.

POSTFIX
operand output; operator pop right, pop left, compute, push.

PREFIX
reverse; swap parentheses; postfix; reverse.

PRECEDENCE
parentheses $>$ exponent $>$ $*/\%$ $>$ $+-$.

SPARSE
store nonzero $(row, column, value)$ terms.

POINTER
`&` address; `*` dereference.

HEAP
malloc/calloc/realloc/free; check NULL.

STORAGE
auto/register/static/extern.

TRAPS
operand order; dangling pointers; wasted linear-queue space.

Source Scope

Synthesized from the supplied PDFs on stack applications, arrays, sparse matrices, polynomial operations, queues, pointers, dynamic memory allocation, and C storage classes. Expression conversion and sparse matrices are intentionally expanded; remaining topics are condensed for revision.

Understand the mechanism first, then memorize the exam wording.